

COMPLETE SQL MASTER GUIDE & STUDY REFERENCE NOTES

Combined Course Syllabus, Normalization, Detailed Concepts, & Complete Practice Queries

SECTION 1: SQL Language Commands & Classification (DQL, DDL, DML, DCL, TCL)

SQL (Structured Query Language) commands are divided into 5 core sub-languages based on their function:

1. DQL (Data Query Language): Used to retrieve records from database tables. Command: SELECT.
2. DDL (Data Definition Language): Used to define, alter, and manage database structures and schemas. Commands: CREATE, ALTER, DROP, TRUNCATE, RENAME.
3. DML (Data Manipulation Language): Used to insert, modify, and delete table data rows. Commands: INSERT, UPDATE, DELETE, MERGE.
4. DCL (Data Control Language): Used to manage user access rights, security, and permissions. Commands: GRANT, REVOKE.
5. TCL (Transaction Control Language): Used to manage database transactions and enforce ACID properties. Commands: COMMIT, ROLLBACK, SAVEPOINT.

SQL Code Examples:

```
-- DDL: Create Table Structure
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary DECIMAL(10, 2),
    hire_date DATE
);

-- DML: Insert and Update Rows
INSERT INTO employees (emp_id, emp_name, salary, hire_date)
VALUES (101, 'Alice Smith', 75000.00, '2023-01-15');

UPDATE employees SET salary = 80000.00 WHERE emp_id = 101;

-- DQL: Retrieve Data
SELECT emp_id, emp_name, salary FROM employees;

-- TCL: Save Transaction Changes
COMMIT;
```

SECTION 2: DDL Operations & Table Management (CREATE, ALTER, DROP, TRUNCATE, RENAME, DESC)

Data Definition Language (DDL) controls database schema structures:

- CREATE TABLE: Defines table columns, data types, and initial constraints.
- DESC / DESCRIBE: Displays the structure (column names, types, nullability) of a table.

- ALTER TABLE: Modifies an existing table structure (ALTER ... MODIFY column, ALTER ... ADD column, ALTER ... DROP COLUMN).
- TRUNCATE TABLE: Removes all data rows from a table permanently. Unlike DELETE, TRUNCATE is a DDL operation, cannot be rolled back, resets storage space, and does not accept a WHERE clause.
- DROP TABLE: Permanently deletes the table schema, indexes, constraints, and all data rows.

SQL Code Examples:

```
-- Create table emp31 with incorrect salary data type
CREATE TABLE emp31 (
    empid INT,
    empname VARCHAR(20),
    dept VARCHAR(20),
    salary DATE,
    doj DATE
);

-- Describe table structure
DESC emp31;

-- Fix column data type using ALTER TABLE MODIFY
ALTER TABLE emp31 MODIFY salary INT;

-- Create table emp32
CREATE TABLE emp32 (
    empid INT,
    empname VARCHAR(20),
    dept VARCHAR(20),
    salary INT,
    doj DATE
);

-- Truncate all rows from emp32 (Fast wipe without logging individual row deletes)
TRUNCATE TABLE emp32;

-- Drop table structure completely
DROP TABLE emp31;
```

SECTION 3: DML Operations & Data Manipulation (INSERT, INSERT ALL, UPDATE, DELETE)

Data Manipulation Language (DML) manages table content:

- INSERT INTO: Inserts single or multiple records into a table.
- INSERT ALL INTO: Inserts rows into multiple tables or multiple rows in a single batch (Oracle syntax).
- UPDATE ... SET: Modifies existing column values based on filtering conditions.
- DELETE FROM: Deletes specific rows matching a WHERE clause or all rows if WHERE is omitted.

SQL Code Examples:

```
-- Single row insertion with full value list
INSERT INTO emp31 VALUES (1, 'Raj', 'account', 654000, '01-Jan-2000');
INSERT INTO emp31 VALUES (2, 'Rohan', 'HR', 78000, '05-Jul-2024');
INSERT INTO emp31 VALUES (NULL, 'pritam', 'cash', 564000, '04-jul-2025');

-- Insertion with explicit column naming
INSERT INTO emp31 (empid, empname, dept, salary, doj)
VALUES (4, 'Rakesh', 'purchase', 32000, '22-september-2000');

-- Batch insertion using INSERT ALL
CREATE TABLE zomato03 (p_id VARCHAR(30), p_name VARCHAR(30), price INT);
INSERT ALL
  INTO zomato03 VALUES('p1', 'Pizza', 200)
  INTO zomato03 VALUES('p2', 'Burger', 350)
  INTO zomato03 VALUES('p3', 'Maggi', 100)
SELECT * FROM dual;

-- Delete specific record
DELETE FROM emp31 WHERE empid = 1;

-- Delete all records from table
DELETE FROM emp31;
```

SECTION 4: Data Querying, Filtering & Sorting (SELECT, WHERE, ORDER BY, DISTINCT)

Data Query Language (DQL) constructs:

- SELECT: Specifies columns to retrieve.
- WHERE: Filters rows based on conditions (=, >, <, BETWEEN, IN, LIKE, IS NULL, AND, OR, NOT).
- ORDER BY: Sorts the output result set in ASC (ascending, default) or DESC (descending) order.
- DISTINCT: Eliminates duplicate rows from the query output.

SQL Code Examples:

```
CREATE TABLE emp66 (emp_id INT, emp_name VARCHAR(20), salary INT, city VARCHAR(20));

INSERT ALL
  INTO emp66 VALUES (1, 'raj', 50000, 'pune')
  INTO emp66 VALUES (2, 'ram', 70000, 'satara')
  INTO emp66 VALUES (2, 'rohan', 55000, 'pune')
  INTO emp66 VALUES (1, 'raj', 50000, 'pune')
SELECT * FROM dual;

-- Select unique emp_id values
SELECT DISTINCT emp_id FROM emp66;

-- Select unique combinations of emp_id and emp_name
SELECT DISTINCT emp_id, emp_name FROM emp66;

-- Filtering with logical conditions and sorting
SELECT * FROM emp66
WHERE salary >= 55000 AND city IN ('pune', 'satara')
ORDER BY salary DESC;
```

SECTION 5: Database Constraints & Data Integrity

Constraints enforce business rules and data consistency across table columns:

1. NOT NULL: Guarantees that a column cannot store NULL values.
2. PRIMARY KEY: Uniquely identifies every row in a table. Implies UNIQUE + NOT NULL. Only one Primary Key allowed per table.
3. UNIQUE: Ensures all values in a column are distinct, but allows NULL values.
4. CHECK: Validates that data entered in a column satisfies a specific boolean expression.
5. DEFAULT: Automatically assigns a default value if no value is explicitly supplied during insert.
6. COMPOSITE PRIMARY KEY: A Primary Key composed of two or more columns combined.
7. Constraint Management: Dropping, enabling, or disabling constraints using ALTER TABLE.

SQL Code Examples:

```
-- 1. NOT NULL Constraint
CREATE TABLE emp40 (id INT NOT NULL, name VARCHAR(20), salary INT);
ALTER TABLE emp40 MODIFY id INT NULL;      -- Modify/Drop NOT NULL
ALTER TABLE emp40 MODIFY id INT NOT NULL; -- Re-apply NOT NULL

-- 2. PRIMARY KEY Constraint
CREATE TABLE emp41 (id INT PRIMARY KEY, name VARCHAR(20), salary INT);
ALTER TABLE emp41 DROP PRIMARY KEY; -- Drop Primary Key

-- 3. UNIQUE Constraint
CREATE TABLE emp44 (id INT UNIQUE, name VARCHAR(40), salary INT UNIQUE);
ALTER TABLE emp44 DROP UNIQUE (salary); -- Drop Unique Constraint

-- 4. CHECK Constraint
CREATE TABLE emp48 (
    id INT,
    name VARCHAR(20),
    salary INT,
    city VARCHAR(20),
    CONSTRAINT ch_city CHECK (city IN ('pune', 'mumbai'))
);

CREATE TABLE emp49 (
    id INT,
    name VARCHAR(20),
    salary INT,
    CONSTRAINT ch_salary CHECK (salary > 50000)
);

-- 5. DEFAULT Constraint
CREATE TABLE emp51 (
    id INT,
    name VARCHAR(20),
    salary INT,
    city VARCHAR(21) DEFAULT 'pune'
);
INSERT INTO emp51 VALUES (2, 'raj', 30000, DEFAULT);

-- 6. COMPOSITE PRIMARY KEY (Multi-column Primary Key)
CREATE TABLE emp53 (
    id INT,
    name VARCHAR(20),
    salary INT,
    city VARCHAR(20),
    CONSTRAINT ch_pk PRIMARY KEY (id, salary)
);
```

SECTION 6: Built-in String & Character Functions

String functions manipulate textual values:

- LENGTH(str): Returns character count.
- CONCAT(str1, str2): Joins string values (can be nested for multiple concatenations).
- UPPER(str) / LOWER(str) / INITCAP(str): Converts casing (Upper, Lower, Title case).
- LPAD(str, len, pad) / RPAD(str, len, pad): Left or right pads string to target length.

- REPLACE(str, search, replace): Replaces substring occurrences (can replace 1 character with multiple characters).
- TRANSLATE(str, from_chars, to_chars): Translates characters 1-to-1 matching positionally.

SQL Code Examples:

```
CREATE TABLE emp34 (id INT, first_name VARCHAR(20), last_name VARCHAR(20), salary INT, doj DATE);
INSERT INTO emp34 VALUES (1, 'rohan', 'kumar', 52000, '01-Jan-2000');
INSERT INTO emp34 VALUES (2, 'priya', 'sharma', NULL, '06-Mar-2004');

-- LENGTH Function
SELECT first_name, LENGTH(first_name) FROM emp34;
SELECT * FROM emp34 WHERE LENGTH(first_name) = 4;

-- CONCAT Function
SELECT CONCAT('Raj', 'Rakesh') FROM dual;
SELECT CONCAT(first_name, CONCAT(' ', last_name)) AS full_name FROM emp34;

-- Casing Functions
SELECT UPPER(first_name), LOWER(last_name), INITCAP(first_name) FROM emp34;
UPDATE emp34 SET first_name = INITCAP(first_name), last_name = INITCAP(last_name);

-- LPAD & RPAD
SELECT LPAD('Hello', 11, 'Rohan ') FROM dual;
SELECT LPAD(first_name, LENGTH(first_name) + LENGTH(last_name), last_name) FROM emp34;

-- REPLACE vs TRANSLATE Comparison
SELECT REPLACE('sanjay', 's', 'w') FROM dual;           -- Output: Wanjay
SELECT REPLACE('sanjay', 's', 'mumbai') FROM dual;     -- Output: mumbaianjay (Replaces 1 char with s)

SELECT TRANSLATE('sanjay', 's', 'w') FROM dual;        -- Output: Wanjay
SELECT TRANSLATE('sanjay', 'sa', 'wk') FROM dual;      -- Output: wknjay (s->w, a->k)
```

SECTION 7: Numeric & Mathematical Functions

Mathematical functions compute numeric outputs:

- MOD(n, m): Calculates remainder of n divided by m. Useful for identifying EVEN rows (MOD(id, 2) = 0) or ODD rows (MOD(id, 2) = 1).
- ROUND(n, [d]): Rounds numeric value to nearest integer or specified decimal places.
- TRUNC(n, [d]): Truncates digits without rounding.
- SQRT(n): Returns square root.
- GREATEST(val1, val2, ...): Returns highest value among parameters.
- LEAST(val1, val2, ...): Returns lowest value among parameters.

SQL Code Examples:

```
-- MOD function and Even/Odd record filtering
SELECT MOD(10, 2) FROM dual; -- Returns 0
SELECT MOD(10, 3) FROM dual; -- Returns 1

-- Select all EVEN rows
SELECT * FROM emp34 WHERE MOD(id, 2) = 0;

-- Select all ODD rows
SELECT * FROM emp34 WHERE MOD(id, 2) = 1;

-- ROUND and TRUNC
SELECT ROUND(10.24) FROM dual; -- 10
SELECT ROUND(10.50) FROM dual; -- 11
SELECT TRUNC(12.24) FROM dual; -- 12

-- GREATEST and LEAST
SELECT GREATEST(10, 20, 30) FROM dual; -- 30
SELECT GREATEST('varsha', 'gaury', 'rohan') FROM dual; -- 'varsha'
SELECT LEAST(10, 20, 30) FROM dual; -- 10
```

SECTION 8: Date, Timestamp & Time Arithmetic Functions

Date/Time processing functions:

- SYSDATE / SYSTIMESTAMP: Returns current system date and detailed timestamp.
- TRUNC(date): Strips time portion, resetting date to 00:00:00.
- ADD_MONTHS(date, n): Adds or subtracts n months from date.
- LAST_DAY(date): Returns final calendar day of specified date's month.
- MONTHS_BETWEEN(date1, date2): Calculates exact difference in months between two dates.
- Age Calculation: MONTHS_BETWEEN(SYSDATE, dob) / 12 calculates age in years.

SQL Code Examples:

```
CREATE TABLE emp36 (id INT, name VARCHAR(20), salary INT, city VARCHAR(20), doj DATE, dob DATE);
INSERT INTO emp36 VALUES (1, 'rohan', 50000, 'pune', '02-feb-2005', '01-Jan-1995');
INSERT INTO emp36 VALUES (2, 'raj', 65980, 'ajamgarh', '02-mar-1985', '01-Jan-1970');

-- System date & Date arithmetic
SELECT ADD_MONTHS(SYSDATE, 1) FROM dual; -- 1 month in future
SELECT ADD_MONTHS(SYSDATE, -1) FROM dual; -- 1 month in past
SELECT LAST_DAY(SYSDATE) FROM dual; -- Last day of current month

-- Calculate Age from DOB
ALTER TABLE emp36 ADD age INT;
UPDATE emp36 SET age = MONTHS_BETWEEN(SYSDATE, dob) / 12;

SELECT id, name, dob, age FROM emp36;
```

SECTION 9: Null Handling Functions (NVL, NVL2, COALESCE, NULLIF)

Functions for handling NULL values:

- NVL(expr1, replace_with): Replaces NULL value with specified replacement value. Crucial in aggregate operations like AVG(NVL(salary, 0)) to avoid skipping NULL rows.
- NVL2(expr1, val_if_not_null, val_if_null): Evaluates expr1; if NOT NULL returns val_if_not_null, if NULL returns val_if_null.
- COALESCE(val1, val2, ...): Returns first non-null expression in parameter list.
- NULLIF(val1, val2): Returns NULL if val1 equals val2, otherwise returns val1.

SQL Code Examples:

```
CREATE TABLE emp37 (id INT, name VARCHAR(20), salary INT, city VARCHAR(20), course VARCHAR(30), certification VARCHAR(30));
INSERT INTO emp37 VALUES (1, 'rohan', 250000, 'pune', 'sql', 'LOTUSGRADE');
INSERT INTO emp37 VALUES (5, 'simran', NULL, 'delhi', 'snowflake', 'IBM');
INSERT INTO emp37 VALUES (7, 'puneet', 56210, 'kolhapur', 'sql', NULL);

-- Basic NVL
SELECT NVL(NULL, 100) FROM dual; -- 100
SELECT AVG(NVL(salary, 0)) FROM emp37;

-- Replace NULL certification values
UPDATE emp37 SET certification = NVL(certification, 'sql');

-- NVL2 Example (Primary contact selection)
CREATE TABLE emp38 (id INT, name VARCHAR(20), contact1 VARCHAR(30), contact2 VARCHAR(30));
INSERT INTO emp38 VALUES (1, 'rakesh', '9860546231', '9654789654');
INSERT INTO emp38 VALUES (3, 'priya', NULL, '9365401524');

SELECT id, name, NVL2(contact1, contact1, contact2) AS primary_contact FROM emp38;
```

SECTION 10: Basic Aggregate Functions & Subqueries

Aggregate calculations across multiple rows:

- MAX(), MIN(), AVG(), SUM(), COUNT()

Subquery Patterns:

- Max / Min Salary Query: WHERE salary = (SELECT MAX(salary) FROM table)
- Second Highest Salary: SELECT MAX(salary) FROM table WHERE salary < (SELECT MAX(salary) FROM table)
- Complete Record for 2nd Highest Salary: Subquery matching 2nd max salary value.

SQL Code Examples:

```
CREATE TABLE emp33 (empid INT, empname VARCHAR(20), dept VARCHAR(20), salary INT, doj DATE);
INSERT INTO emp33 VALUES (1, 'Raj', 'account', 654000, '01-Jan-2000');
INSERT INTO emp33 VALUES (2, 'Rohan', 'HR', 78000, '05-Jul-2024');
INSERT INTO emp33 VALUES (NULL, 'pritam', 'cash', 564000, '04-jul-2025');
INSERT INTO emp33 VALUES (5, 'priya', 'bank', NULL, '04-jul-2025');

-- Max & Min
SELECT MAX(salary), MIN(salary) FROM emp33;

-- Fetch record with Maximum Salary
SELECT * FROM emp33 WHERE salary = (SELECT MAX(salary) FROM emp33);

-- Find 2nd Highest Salary Value
SELECT MAX(salary) FROM emp33 WHERE salary < (SELECT MAX(salary) FROM emp33);

-- Fetch Complete Record with 2nd Highest Salary
SELECT * FROM emp33
WHERE salary = (
    SELECT MAX(salary) FROM emp33
    WHERE salary < (SELECT MAX(salary) FROM emp33)
);
```

SECTION 11: Aggregation, Grouping & Advanced Grouping (ROLLUP, CUBE, HAVING)

Grouping & Advanced Subtotaling:

- GROUP BY: Groups identical column values together for aggregation.
- HAVING: Filters aggregated groups (unlike WHERE which filters rows before aggregation).
- ROLLUP(col): Computes hierarchical subtotals and grand totals.
- CUBE(col1, col2): Computes all cross-tabulation subtotal combinations.

SQL Code Examples:

```
CREATE TABLE emp56 (emp_id INT, name VARCHAR(20), salary INT, city VARCHAR(20), dept VARCHAR(20))
INSERT INTO emp56 VALUES (1, 'ram', 50000, 'pune', 'hr');
INSERT INTO emp56 VALUES (2, 'rohan', 55000, 'pune', 'account');
INSERT INTO emp56 VALUES (3, 'priya', 52000, 'satara', 'hr');
INSERT INTO emp56 VALUES (5, 'raj', 60000, 'pune', 'account');

-- GROUP BY single & multiple columns
SELECT dept, COUNT(*) AS total_emp, SUM(salary) FROM emp56 GROUP BY dept;
SELECT dept, city, COUNT(*), SUM(salary) FROM emp56 GROUP BY dept, city;

-- ROLLUP Subtotals + Grand Total
SELECT NVL(dept, 'TOTAL') AS Dept, COUNT(*), SUM(salary)
FROM emp56
GROUP BY ROLLUP(dept);

-- CUBE All Combinations
SELECT dept, city, COUNT(*), SUM(salary)
FROM emp56
GROUP BY CUBE(dept, city);

-- HAVING clause filter
SELECT dept, SUM(salary) AS total_salary
FROM emp56
GROUP BY dept
HAVING SUM(salary) > 100000;
```

SECTION 12: Conditional Categorization (CASE Statement & DECODE)

Conditional Logic Constructs:

- DECODE(expr, search1, res1, search2, res2, ..., [default]): Compact Oracle equality evaluation.
- CASE WHEN condition THEN result ... ELSE default END: Standard ANSI conditional logic.
- Conditional Pivot Aggregations: SUM(CASE WHEN condition THEN 1 ELSE 0 END) counts rows satisfying criteria per group.

SQL Code Examples:

```
CREATE TABLE emp67 (id INT, name VARCHAR(50), salary INT, gender VARCHAR(8));
INSERT INTO emp67 VALUES (1, 'raju', 65000, 'M'), (3, 'priya', 65000, 'F'), (4, 'kiran', 45000, 'M');

-- DECODE Function
SELECT name, DECODE(gender, 'M', 'Male', 'F', 'Female', 'not found') AS gender_full FROM emp67;

-- CASE Expression
SELECT name, salary,
CASE
    WHEN salary > 50000 THEN 'high level'
    WHEN salary > 30000 THEN 'medium level'
    ELSE 'low level'
END AS salary_level FROM emp67;

-- Pivot Count by Salary Tier per Dept
CREATE TABLE emp68 (id INT, name VARCHAR(50), salary INT, dept VARCHAR(20));
SELECT dept,
    SUM(CASE WHEN salary BETWEEN 1 AND 30000 THEN 1 ELSE 0 END) AS low_level,
    SUM(CASE WHEN salary BETWEEN 30001 AND 60000 THEN 1 ELSE 0 END) AS med_level,
    SUM(CASE WHEN salary > 60000 THEN 1 ELSE 0 END) AS high_level
FROM emp68
GROUP BY dept;
```

SECTION 13: SET Operators (UNION, UNION ALL, INTERSECT, MINUS)

Set Operators combine multi-query output sets. Requirements: identical column count and matching data types.

- UNION: Combines query results, removes duplicate rows.
- UNION ALL: Combines query results, retains duplicate rows (faster).
- INTERSECT: Returns only rows present in both result sets.
- MINUS: Returns rows present in first query but absent from second query.

SQL Code Examples:

```
CREATE TABLE e101 (id INT, name VARCHAR(30), salary INT, city VARCHAR(30));
CREATE TABLE e102 (id INT, name VARCHAR(33), salary INT, city VARCHAR(40));

-- UNION (Distinct rows)
SELECT * FROM e101 UNION SELECT * FROM e102;

-- UNION ALL (All rows including duplicates)
SELECT * FROM e101 UNION ALL SELECT * FROM e102;

-- INTERSECT (Matching rows in both)
SELECT * FROM e101 INTERSECT SELECT * FROM e102;

-- MINUS (Rows in e101 but not e102)
SELECT * FROM e101 MINUS SELECT * FROM e102;
```

SECTION 14: Relational Schemas, Foreign Keys & Star Schema

Relational Architecture & Star Schema Design:

- Foreign Keys enforce referential integrity between parent primary key and child table.

- Star Schema: Multi-dimensional database design featuring central Fact tables (containing quantitative sales measures) surrounded by Dimension tables (descriptive entities like item, location, branch, time).

SQL Code Examples:

```
-- Dimension Tables
CREATE TABLE time (time_id INT PRIMARY KEY, day INT, month VARCHAR(15), quarter VARCHAR(2), year INT);
CREATE TABLE item (item_key INT PRIMARY KEY, item_name VARCHAR(100), brand VARCHAR(50), category VARCHAR(50));
CREATE TABLE location (location_id INT PRIMARY KEY, name VARCHAR(100), state VARCHAR(50));
CREATE TABLE branch (branch_id INT PRIMARY KEY, branch_nm VARCHAR(100));

-- Sales Fact Table with Foreign Keys referencing Dimensions
CREATE TABLE sales_fact (
    item_key INT,
    time_id INT,
    branch_id INT,
    location_id INT,
    qty_sold INT,
    amt_sold INT,
    FOREIGN KEY (item_key) REFERENCES item(item_key),
    FOREIGN KEY (time_id) REFERENCES time(time_id),
    FOREIGN KEY (branch_id) REFERENCES branch(branch_id),
    FOREIGN KEY (location_id) REFERENCES location(location_id)
);

-- Querying Star Schema
SELECT t.year, i.item_name, l.state, s.qty_sold, s.amt_sold
FROM sales_fact s
JOIN time t ON s.time_id = t.time_id
JOIN item i ON s.item_key = i.item_key
JOIN location l ON s.location_id = l.location_id;
```

SECTION 15: Database Design & Normalization (1NF, 2NF, 3NF, BCNF)

Database Normalization structures table columns to eliminate data redundancy and data modification anomalies:

1. Insertion Anomaly: Inability to insert data without inserting redundant details.
2. Update Anomaly: Data inconsistency when modifying duplicate row values.
3. Deletion Anomaly: Accidental data loss when deleting related records.

Normal Forms Rules:

- 1NF (First Normal Form): Column values must be atomic (indivisible single values); no repeating groups or comma-separated lists.
- 2NF (Second Normal Form): Table must be in 1NF + no partial functional dependencies (all non-key columns must fully depend on the entire composite primary key).
- 3NF (Third Normal Form): Table must be in 2NF + no transitive functional dependencies (non-key columns must not depend on other non-key columns).
- BCNF (Boyce-Codd Normal Form): Strict version of 3NF where every functional determinant ($X \rightarrow Y$) must be a Candidate Key.

SQL Code Examples:

```
-- Unnormalized / Denormalized Table (Contains Redundancies & Anomalies)
-- Student_Course(student_id, student_name, course_id, course_name, instructor_name)

-- Normalized Database Design (3NF Refactored Tables)

-- Table 1: Students Entity
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL
);

-- Table 2: Courses Entity
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    instructor_name VARCHAR(100) NOT NULL
);

-- Table 3: Student-Course Junction Table (Resolves Many-to-Many relationship into 3NF)
CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    enrollment_date DATE DEFAULT SYSDATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

SECTION 16: SQL Joins (Inner, Left, Right, Full, Self, Cross)

Combining data across multiple tables:

- INNER JOIN: Returns matching records in both tables.
- LEFT JOIN: Returns all left table records + matched right table records. Unmatched right side shows NULL.
- RIGHT JOIN: Returns all right table records + matched left table records.
- FULL JOIN: Returns all rows from both tables.
- SELF JOIN: Table joined with itself to model hierarchy (e.g. Employee vs Manager).
- CROSS JOIN: Cartesian product of both tables.

SQL Code Examples:

```
CREATE TABLE product (prod_id INT, prod_name VARCHAR(50), prod_price INT);
CREATE TABLE customer (cust_id INT, cust_name VARCHAR(50), city VARCHAR(50));

-- INNER JOIN
SELECT p.*, c.* FROM product p INNER JOIN customer c ON p.prod_id = c.cust_id;

-- LEFT JOIN (Find products NEVER sold)
SELECT p.*, c.* FROM product p LEFT JOIN customer c ON p.prod_id = c.cust_id WHERE c.cust_id IS NULL;

-- RIGHT JOIN (Find customers who NEVER purchased products)
SELECT p.*, c.* FROM product p RIGHT JOIN customer c ON p.prod_id = c.cust_id WHERE p.prod_id IS NULL;

-- FULL JOIN
SELECT p.*, c.* FROM product p FULL JOIN customer c ON p.prod_id = c.cust_id;

-- SELF JOIN (Compare Employee Salary vs Manager Salary)
CREATE TABLE supply1 (em_id INT, em_name VARCHAR(20), salary INT, ma_id INT);
SELECT e.em_id, e.em_name, e.salary, m.ma_id, m.salary AS mgr_salary
FROM supply1 e INNER JOIN supply1 m ON e.ma_id = m.em_id
WHERE e.salary > m.salary;

-- CROSS JOIN
SELECT s.*, v.* FROM product s CROSS JOIN customer v;
```

SECTION 17: Pseudo-columns (ROWID, ROWNUM), Inline Views & Duplicate Removal

System Pseudo-columns:

- ROWID: Unique physical hex storage address of a row in disk storage.
- ROWNUM: Sequential integer assigned to rows as they satisfy query criteria.
- Top-N Query Pattern: Must sort in an Inline View first before filtering ROWNUM <= N.
- Duplicate Removal Pattern: Delete rows where ROWID is NOT IN (SELECT MAX(ROWID) FROM table GROUP BY duplicate_cols).

SQL Code Examples:

```
CREATE TABLE emp71 (id INT, name VARCHAR(20), salary VARCHAR(20), city VARCHAR(20));

-- View ROWID and ROWNUM
SELECT ROWID, ROWNUM, t.* FROM emp71 t;

-- Fetch Top 3 Highest Salary records (Inline View)
SELECT * FROM (
    SELECT * FROM emp68 ORDER BY salary DESC
) WHERE ROWNUM <= 3;

-- Find Duplicate Rows
SELECT * FROM emp68
WHERE ROWID NOT IN (
    SELECT MAX(ROWID) FROM emp68 GROUP BY id
);

-- Delete Duplicate Rows keeping 1 unique row instance
DELETE FROM emp68
WHERE ROWID NOT IN (
    SELECT MAX(ROWID) FROM emp68 GROUP BY id
);
```

SECTION 18: Database Sequences

Sequences generate sequential numeric values:

- CREATE SEQUENCE seq_name START WITH 1 INCREMENT BY 1 MAXVALUE n
- seq_name.NEXTVAL: Generates and returns next sequence number.
- seq_name.CURRVAL: Returns current sequence value in session.

SQL Code Examples:

```
CREATE TABLE emp72 (id INT, name VARCHAR(20), salary INT, city VARCHAR(20));

-- Create Sequence
CREATE SEQUENCE s1
START WITH 1
INCREMENT BY 1;

-- Insert records using Sequence
INSERT INTO emp72 VALUES (s1.NEXTVAL, 'raj', 50000, 'satara');
INSERT INTO emp72 VALUES (s1.NEXTVAL, 'rohan', 55000, 'pune');

-- Create Sequence with Maxvalue
CREATE SEQUENCE s3 START WITH 2 INCREMENT BY 2 MAXVALUE 10;
```

SECTION 19: Views, Materialized Views, Synonyms & Copying Tables

Database View & Storage Abstractions:

- Simple View: Virtual table based on SELECT query. DML executed on view updates underlying base table.
- Materialized View: Physically stores and caches query results on disk for fast analytical access.
- Synonym: Permanent alias name for a table, view, or database object.

- Copy Table: CREATE TABLE new_table AS SELECT ... copies table definition and data rows.

SQL Code Examples:

```
-- 1. Create Simple View
CREATE VIEW emp78 AS SELECT * FROM emp77;
INSERT INTO emp78 VALUES (8, 'kiran', 56000, 'pune');

-- 2. Create Materialized View
CREATE MATERIALIZED VIEW emp79 AS SELECT * FROM emp77;

-- 3. Create Synonym
CREATE SYNONYM emp80 FOR emp77;

-- 4. Copy Table
CREATE TABLE emp84 AS SELECT * FROM emp83;
CREATE TABLE emp85 AS SELECT name, city FROM emp83;
```

SECTION 20: Analytical & Window Functions (RANK, DENSE_RANK, ROW_NUMBER, PARTITION BY)

Window functions compute calculations across table rows related to current row:

- ROW_NUMBER(): Assigns unique sequential integers (1, 2, 3, 4...).
- RANK(): Assigns rank with gaps for ties (1, 2, 2, 4...).
- DENSE_RANK(): Assigns rank without gaps for ties (1, 2, 2, 3...).
- PARTITION BY col: Groups calculations per partition.
- Window Aggregates: SUM(salary) OVER (PARTITION BY dept) computes department totals without collapsing individual row detail.

SQL Code Examples:

```
CREATE TABLE emp90 (id INT, name VARCHAR(20), salary INT, city VARCHAR(20), dept VARCHAR(20), doj DATE);
INSERT INTO emp90 VALUES (1, 'raj', 50000, 'pune', 'account', '21-sep-2000');
INSERT INTO emp90 VALUES (2, 'ram', 60000, 'mumbai', 'account', '22-sep-2000');
INSERT INTO emp90 VALUES (3, 'rohan', 50000, 'pune', 'hr', '30-sep-2000');

-- Compare RANK, DENSE_RANK, ROW_NUMBER
SELECT e.*,
       RANK() OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM emp90 e;

-- Partition by Dept (Fetch highest earner per department)
SELECT * FROM (
    SELECT e.*, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn
    FROM emp90 e
) WHERE rn = 1;

-- Window Aggregate Total
SELECT e.*, SUM(salary) OVER (PARTITION BY dept) AS dept_running_total FROM emp90 e;
```

SECTION 21: Security, Privileges & Database Administration

User Management & Data Administration:

- CREATE USER username IDENTIFIED BY password;
- CREATE TABLESPACE tablespace_name DATAFILE 'path' SIZE 100M;
- GRANT privilege TO username / REVOKE privilege FROM username;
- WITH GRANT OPTION: Allows user to grant their received privilege to other database users.
- CREATE ROLE role_name: Packages multiple permissions into a single manageable role.
- Table Partitioning: Range, List, Hash partitioning for massive tables.
- SQL*Loader: High-speed Oracle utility for bulk loading external data files (CSV).

SQL Code Examples:

```
-- Create Storage Tablespace and Database User
CREATE TABLESPACE user_data DATAFILE 'userdata01.dbf' SIZE 100M;

CREATE USER analyst_user IDENTIFIED BY SecurePass123!
DEFAULT TABLESPACE user_data;

-- Create Role and Grant Privileges
CREATE ROLE data_analyst_role;
GRANT SELECT, INSERT ON employees TO data_analyst_role;
GRANT data_analyst_role TO analyst_user;

-- Grant with Grant Option & Revoke
GRANT SELECT ON employees TO analyst_user WITH GRANT OPTION;
REVOKE SELECT ON employees FROM analyst_user;
```

SECTION 22: Real-World Business Intelligence Queries

Practical Multi-concept Business Queries:

- Categorizing customer visit frequency.
- Calculating customer spend and assigning loyalty card tiers (Platinum, Golden, Silver).
- Finding latest transaction dates per customer.

SQL Code Examples:

```
-- Customer Visit Categorization
SELECT cust_id, COUNT(*) AS visits,
CASE
    WHEN COUNT(*) >= 3 THEN 'Regular Customer'
    WHEN COUNT(*) >= 2 THEN 'Ordinary Customer'
    ELSE 'Casual Customer'
END AS cust_visit_category
FROM customer03
GROUP BY cust_id;

-- Customer Spend & Loyalty Card Classification
SELECT c.cust_name, COUNT(*) AS visits, SUM(z.price) AS total_sales,
CASE
    WHEN COUNT(*) >= 3 AND SUM(z.price) >= 400 THEN 'Platinum'
    WHEN COUNT(*) >= 2 AND SUM(z.price) >= 200 THEN 'Golden'
    ELSE 'Silver'
END AS card_category
FROM customer03 c
INNER JOIN zomato03 z ON c.p_id = z.p_id
GROUP BY c.cust_name;

-- Latest Purchase Date per Customer
SELECT c.cust_name, MAX(c.purchase_date) AS latest_purchase_date
FROM customer03 c
GROUP BY c.cust_name;
```